Guía para usar los archivos Python

2 de octubre de 2026

Qué hace cada archivo y quién lo utiliza

Archivo	Responsable y laptop	Función	Forma de uso
app.py	TI, laptop del servidor Ubuntu	Abre la aplicación web, conecta el inicio de sesión OneLogin y recibe altas, cambios y bajas	Se inicia mediante Gunicorn
store.py	TI, misma carpeta del servidor	Guarda usuarios, permisos y eventos en SQLite	Lo utiliza app.py; no se ejecuta por separado
test_store.py	TI y Auditoría	Comprueba siete casos de permisos y bajas	Se ejecuta con unittest
registrar_tarjetas.py	Seguridad Física, laptop conectada por USB a la placa	Guarda las lecturas de tarjetas en un CSV con hora	Se ejecuta con Python indicando el puerto

Los archivos Python están en Implementacion/app/ y Implementacion/rfid/. El archivo control_acceso.ino se carga a la placa desde Arduino IDE; no es Python. Frigate se inicia con Docker Compose, no con estos scripts. No hace falta ejecutar Python en las 31 laptops.

1. Preparar las carpetas

Descomprimir el paquete completo. Copiar la carpeta app con todos sus archivos, incluido .env.example, a ~/lab-runtime/app/ en la laptop TI. Copiar rfid a ~/lab-runtime/rfid/ en la laptop de Seguridad Física. No mezclar ambas carpetas ni renombrar store.py: la aplicación lo busca por ese nombre.

Los comandos del servidor se ejecutan en Ubuntu 24.04. En Windows, usar la máquina virtual Ubuntu explicada en la guía TI. Los comandos para el registrador RFID sí incluyen una alternativa Windows.

2. Instalar Python para el servidor

En una terminal Ubuntu:

```
sudo apt update
sudo apt install -y python3 python3-venv python3-pip
python3 --version
cd ~/lab-runtime/app
ls -a
python3 -m venv .venv
source .venv/bin/activate
python -m pip install -r requirements.txt
```

ls -a debe mostrar app.py, store.py, test_store.py, requirements.txt y .env.example. El entorno .venv mantiene las dependencias de este proyecto separadas. Activarlo cada vez que abran una terminal para trabajar en la aplicación. No usar sudo pip.

3. Ejecutar las pruebas antes de conectar OneLogin

En la carpeta app, con el entorno activado:

```
python -m unittest -v test_store.py
```

El resultado esperado es Ran 7 tests y OK. Las pruebas comprueban que mantenimiento no tiene administración, un cambio de rol retira el permiso anterior, una cuenta inactiva no puede actuar, el texto false se interpreta correctamente, un rol desconocido no permite acceso, una identidad no se vincula a otro identificador y una cuenta eliminada pierde acceso. Usan una base temporal y no crean usuarios en OneLogin.

Si aparece FAILED o ERROR, leer el nombre de la prueba que falló y entregar el mensaje a TI. No cambiar la prueba para que pase. Estas pruebas no certifican la conexión con OneLogin, la grabación de video ni el lector físico.

4. Configurar las variables de la aplicación

```
cp .env.example .env
chmod 600 .env
python -c 'import secrets; print(secrets.token_urlsafe(48))'
python -c 'import secrets; print(secrets.token_urlsafe(48))'
nano .env
```

Copiar el primer resultado a APP_SECRET y el segundo a SCIM_TOKEN. Son valores distintos. Completar las demás variables con IAM:

Variable	Qué poner
APP_SECRET	Secreto generado para proteger la sesión

Variable	Qué poner
SCIM_TOKEN	Secreto generado; se configura también en el conector de aprovisionamiento de OneLogin
APP_BASE_URL	Dirección HTTPS real del túnel, sin barra final
OIDC_CLIENT_ID	Identificador que muestra el conector OIDC
OIDC_CLIENT_SECRET	Secreto de ese conector
OIDC_METADATA_URL	Dirección de configuración del tenant OneLogin, copiada de la configuración real
APP_DB	data/lab.sqlite, si se mantiene la estructura propuesta

El archivo se carga desde una terminal de shell. Mantener valores sin espacios o encerrarlos entre comillas simples. No pegar comandos, sustituciones de shell ni contenido ajeno al proyecto dentro de .env. No adjuntar ese archivo a evidencias.

5. Abrir el túnel y arrancar el servidor

Primero completar la instalación Docker de la guía TI. En una terminal independiente:

```
sudo docker run --rm --network host cloudflare/cloudflared:latest tunnel --url http://127.0.0.1:8000
```

Copiar la dirección HTTPS que aparece. Actualizar APP_BASE_URL en .env. IAM actualiza también la dirección de retorno OIDC y la base SCIM como indica su guía. Mantener esa terminal abierta.

En otra terminal:

```
cd ~/lab-runtime/app
source .venv/bin/activate
set -a
source .env
set +a
mkdir -p data
gunicorn --workers 1 --bind 127.0.0.1:8000 --access-logfile - --error-logfile - app:app
```

app:app significa usar el objeto web llamado app dentro de app.py. No ejecutar store.py ni iniciar el proyecto con python app.py: el archivo no incluye un servidor de arranque directo. Gunicorn mantiene el servicio funcionando mientras la terminal permanezca abierta.

En una tercera terminal:

```
curl http://127.0.0.1:8000/health
```

Debe devolver {”status”:.ºk”}. Abrir la dirección HTTPS desde otra laptop. El primer botón conduce a OneLogin. Un usuario debe estar autenticado y además existir como cuenta activa aprovisionada en la aplicación. Si aparece 403, IAM revisa su cuenta SCIM, correo y rol; no se habilita un acceso local alternativo para saltar la prueba.

6. Qué guardar como evidencia

Guardar la salida de las siete pruebas, la comprobación de salud y capturas de acceso permitido y rechazado. Los registros están en data/lab.sqlite; la aplicación muestra los eventos desde / registros a un usuario con permiso de auditor o administrador. Los roles que reconoce son admin, mantenimiento, auditor, consulta y sin_acceso.

El archivo SQLite no se abre como Excel. Antes de copiarlo como respaldo, detener Gunicorn con Ctrl+C y copiar también cualquier archivo asociado de SQLite que exista en la misma carpeta. Para registrar dependencias:

```
python -m pip freeze > versiones_instaladas.txt
```

No eliminar la base para resolver un error: se perderían cuentas y eventos. Los botones de mantenimiento y administración modifican el contador y estado del recurso de aplicación; no ejecutan comandos del sistema operativo.

7. Preparar el registrador RFID en Ubuntu

Confirmar primero placa y lector. El ejemplo incluido corresponde a Arduino Uno y MFRC522. Seguridad Física debe cargar control_acceso.ino, reemplazar los cuatro UID y probar el formato de las lecturas antes de ejecutar el registrador. El CSV registra decisiones del microcontrolador; no decide por sí mismo quién entra.

```
cd ~/lab-runtime/rfid
python3 -m venv .venv
source .venv/bin/activate
python -m pip install pyserial
python -m serial.tools.list_ports -v
```

Anotar el puerto que aparece al conectar la placa, por ejemplo /dev/ttyACM0. Cerrar el monitor serie de Arduino IDE: solo un programa debe abrir el puerto. Si Ubuntu rechaza el acceso por permisos:

```
sudo usermod -aG dialout ”$USER”
```

Cerrar completamente la sesión del usuario y volver a entrar para aplicar el grupo. Volver a activar el entorno. No ejecutar el registrador con sudo como solución habitual.

8. Preparar el registrador RFID en Windows

Instalar Python desde la página oficial. Abrir PowerShell y comprobar el lanzador py. Copiar la carpeta RFID, por ejemplo a C:\lab-runtime\rfid, y ejecutar:

```
cd C:\lab-runtime\rfid
py --version
py -m venv .venv
.\.venv\Scripts\python.exe -m pip install pyserial
.\.venv\Scripts\python.exe -m serial.tools.list_ports -v
```

Esta alternativa llama directamente al Python del entorno y no necesita modificar la política de ejecución de PowerShell. Confirmar el puerto real, por ejemplo COM3, en el listado o en el Administrador de dispositivos. Cerrar el monitor serie de Arduino IDE.

9. Registrar entrada y salida

Ubuntu, reemplazando el puerto si corresponde:

```
cd ~/lab-runtime/rfid
source .venv/bin/activate
python registrar_tarjetas.py --puerto /dev/ttyACM0 --archivo eventos_rfid.csv
```

Windows:

```
cd C:\lab-runtime\rfid
.\.venv\Scripts\python.exe registrar_tarjetas.py --puerto COM3 --archivo eventos_rfid.csv
```

Dejar la terminal abierta. Escribir E y pulsar Enter para entrada; escribir S y pulsar Enter para salida. Esperar el mensaje de modo solicitado antes de pasar la tarjeta. Este es un único lector con dirección seleccionada por el operador, no un sistema que detecta automáticamente por qué lado camina una persona.

La placa debe enviar una línea como ENTRADA,AB12CD34,PERMITIDO,LAB-INGENIERO. El registrador acepta cuatro campos, movimiento ENTRADA/SALIDA y resultado PERMITIDO/DENEGADO. Otros mensajes se muestran pero no se guardan como evento. El UID del ejemplo es ficticio.

El CSV contiene hora UTC, hora local, movimiento, UID, resultado y empleado. Si el archivo ya existe, agrega filas sin borrar lo anterior. La hora es la recepción en la laptop: sincronizar su reloj antes del ensayo. El arranque serie puede reiniciar una placa Uno; esperar a que esté lista antes de pasar tarjetas.

Detener con Ctrl+C antes de abrir el CSV en Excel o copiarlo al SOC. Hacer tres comprobaciones: tarjeta permitida, tarjeta denegada y salida de una tarjeta permitida. Confirmar las tres filas y relacionarlas con el video del mismo momento.

10. Apagar y volver a iniciar

Para la aplicación, usar Ctrl+C en Gunicorn y luego en la terminal del túnel. Para el lector, Ctrl+C en el registrador. La base y el CSV permanecen guardados. La próxima sesión exige activar el entorno, cargar .env y volver a iniciar Gunicorn. Si la dirección del túnel cambió, actualizar los tres lugares indicados antes de probar OneLogin.

Problemas frecuentes

Mensaje o situación	Solución
ModuleNotFoundError: flask o authlib	Activar .venv de la aplicación e instalar requirements.txt
ModuleNotFoundError: serial	Instalar pyserial en el entorno RFID; no instalar un paquete llamado serial
KeyError: APP_SECRET u otra variable	Cargar .env con set -a, source .env, set +a
Error de secretos o URL HTTPS	Sustituir los ejemplos, usar secretos largos y la URL HTTPS real
Address already in use	Hay otro servidor usando el puerto 8000; detener la instancia previa antes de arrancar otra
Error de acceso al puerto serie	Confirmar puerto, permisos y que Arduino IDE u otro proceso no lo tengan abierto
La terminal muestra lecturas pero no hay filas CSV	Revisar el formato de cuatro campos que envía la placa
502 desde el túnel	Comprobar Gunicorn y /health en la laptop TI
403 después del inicio OneLogin	Revisar cuenta aprovisionada activa, correo y rol con IAM
El CSV muestra otra hora	Corregir zona horaria y reloj; no editar la evidencia para simular un evento

Referencias

- Python: instalación y uso.
- Python: entornos virtuales.
- Python: unittest.
- pySerial: listado de puertos.
- Gunicorn: ejecución.
- Guía TI (03_TI.pdf) y guía IAM (04_IAM_OneLogin.pdf).